📋 Таска: Приложение `crackerchat` — модель `Post` и первый эндпоинт API

🎯 Цель

Создать Django-приложение `crackerchat` с моделью `Post`, зарегистрировать её в админке и отдать по REST API два эндпоинта: `GET /api/posts/` (список постов, новые сверху) и `POST /api/posts/` (создать пост). Это разблокирует фронтенд — они смогут получать и отправлять реальные посты вместо мок-данных.

📦 Предварительно

Убедись, что:

- `djangorestframework` установлен (`pip install djangorestframework`), версия зафиксирована в `requirements.txt`;
- `rest_framework` уже добавлен в `INSTALLED_APPS` (из подготовительного этапа);
- виртуальное окружение активировано, `python manage.py runserver` работает;
- ты находишься в папке `backend/`.

🛠 Шаг 1: Создать приложение

Из папки `backend/` (где лежит `manage.py`):

```
python manage.py startapp crackerchat
```

Появится папка `backend/crackerchat/` с заготовкой: `models.py`, `views.py`, `admin.py`, `apps.py`, `migrations/` и т.д.

🛠 Шаг 2: Зарегистрировать в `INSTALLED_APPS`

Открой `backend/config/settings.py`, найди `INSTALLED_APPS` и добавь строку:

```
INSTALLED_APPS = [
    # ...
    "rest_framework",
    "corsheaders",
    "crackerchat",    # ← добавили
]
```

> 💡 Django найдёт приложение по имени папки — `crackerchat`. Внутри есть `apps.py` с классом `CrackerchatConfig`, но в `INSTALLED_APPS` достаточно указать имя пакета.

🛠 Шаг 3: Модель `Post`

Открой `backend/crackerchat/models.py` и замени содержимое на:

```
from django.db import models


class Post(models.Model):
    text = models.TextField(max_length=2000)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        ordering = ["-created_at"]  # новые сверху
        verbose_name = "Пост"
        verbose_name_plural = "Посты"

    def __str__(self):
        return f"#{self.pk} {self.text[:30]}"
```

Что здесь важно:

- `max_length=2000` — ограничение на размер поста. Работает и для `TextField` (валидация на уровне форм/сериализаторов, не на уровне БД).
- `auto_now_add=True` — Django сам проставит дату при создании, менять её вручную нельзя.
- `db_index=True` — индекс по `created_at`. Нам он нужен, потому что сортировка и пагинация будут именно по этому полю. На маленькой таблице не заметно, но закладываем правильную привычку.
- `ordering = ["-created_at"]` — **дефолтная сортировка queryset'а**. Любой `Post.objects.all()` уже будет отсортирован «новые сверху», без `order_by` в каждом view. Это очень удобно и часто забывают.

🛠 Шаг 4: Миграции

```
python manage.py makemigrations crackerchat
python manage.py migrate
```

Ожидаемый вывод:

```
crackerchat/migrations/0001_initial.py
  - Create model Post
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, crackerchat, sessions
Running migrations:
  Applying crackerchat.0001_initial... OK
```

> ⚠️ Если сделать только `migrate` без `makemigrations`, Django создаст таблицы для встроенных приложений, но **не** для `Post`. Таблица появится только после `makemigrations + migrate`.

> Если забыл `makemigrations` — просто выполни его, потом `migrate` ещё раз.

🛠 Шаг 5: Регистрация в админке

Открой `backend/crackerchat/admin.py`:

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("id", "short_text", "created_at")
    ordering = ("-created_at",)
    search_fields = ("text",)

    @admin.display(description="Текст")
    def short_text(self, obj):
        return obj.text[:80]
```

Зачем: через `/admin/` ты и студенты сможете смотреть посты и удалять спам. Пока у нас нет антиспама, это единственный способ модерировать борду.

Создай админа, если ещё не сделал:

```
python manage.py createsuperuser
```

Проверь: открой `http://127.0.0.1:8000/admin/`, залогинься, в разделе «Crackerchat → Посты» должна быть пустая таблица.

🛠 Шаг 6: Сериализатор

Создай файл `backend/crackerchat/serializers.py`:

```
from rest_framework import serializers
from .models import Post


class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = ["id", "text", "created_at"]
        read_only_fields = ["id", "created_at"]

    def validate_text(self, value):
        value = value.strip()
        if not value:
            raise serializers.ValidationError("Пост не может быть пустым.")
        return value
```

- `read_only_fields` — клиент не может подделать `id` или `created_at`; они заполняются сервером.
- `validate_text` — триммит пробелы и отклоняет пустой текст. Без этого можно отправить `{"text": " "}` и получить «пустой» пост.

🛠 Шаг 7: ViewSet

Открой `backend/crackerchat/views.py`:

```
from rest_framework import mixins, viewsets

from .models import Post
from .serializers import PostSerializer


class PostViewSet(
    mixins.ListModelMixin,
    mixins.CreateModelMixin,
    viewsets.GenericViewSet,
):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
```

> 💡 **Почему не `ModelViewSet`?** `ModelViewSet` даёт полный CRUD: `retrieve`, `update`, `partial_update`, `destroy`. Для анонимной борды это **опасно** — кто угодно сможет отредактировать или удалить чужой пост, зная `id`. Мы даём только то, что нужно: **list + create**. Остальные методы вернут `405 Method Not Allowed`. Это и правильно семантически, и защищает от случайных правок.
>
> Сортировка `-created_at` уже применяется автоматически благодаря `Meta.ordering` в модели.

🛠 Шаг 8: URL'ы приложения

Создай `backend/crackerchat/urls.py`:

```
from rest_framework.routers import DefaultRouter

from .views import PostViewSet

router = DefaultRouter()
router.register(r"posts", PostViewSet, basename="post")

urlpatterns = router.urls
```

`DefaultRouter` автоматически создаст маршрут `/posts/` и повесит на него `PostViewSet`. `basename` нужен, потому что у нас нет `queryset` с именем модели в стандартном смысле — можно и опустить (DRF выведет из `queryset.model`), но явное лучше неявного.

🛠 Шаг 9: Подключить в корневых URL'ах

Открой `backend/config/urls.py` и приведи к виду:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("crackerchat.urls")),
]
```

Префикс `api/` — потому что `/posts/` в корне сайта нам не нужен, а `/api/posts/` — понятная и знакомая фронту структура.

🛠 Шаг 10: Настройки DRF (анонимный доступ)

Открой `backend/config/settings.py`, добавь в конец (если ещё нет):

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [],  # анонимная борда — аутентификация не нужна
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.AllowAny",
    ],
}
```

> ⚠️ **Почему пустой `DEFAULT_AUTHENTICATION_CLASSES`?** По умолчанию DRF включает `SessionAuthentication`, которая **требует CSRF-токен** для любого небезопасного метода (POST/PUT/DELETE). Для анонимной борды с `fetch`-запросами это лишнее трение: `curl`-запрос на `POST` без CSRF-токена вернёт `403 CSRF verification failed`. Убираем session-auth — `POST` работает без плясок с токенами. Когда/если добавим аутентификацию — вернём.
>
> `AllowAny` — потому что регистрации нет, все анонимны. Позже, когда будем ограничивать через `AnonRateThrottle` или таймаут, `AllowAny` останется — ограничения будут на других уровнях.

Перезапусти сервер после правок `settings.py` и `INSTALLED_APPS`:

```
python manage.py runserver
```

✅ Проверка

1. GET — список (пустой):

```
curl http://127.0.0.1:8000/api/posts/
```

Ожидается: `[]` (или `{"detail": ...}` ? нет — пустой список, `[]`).

2. POST — создание:

```
curl -X POST http://127.0.0.1:8000/api/posts/ \
  -H "Content-Type: application/json" \
  -d '{"text":"первый пост"}'
```

Ожидается статус `201 Created` и JSON вида:

```
{"id": 1, "text": "первый пост", "created_at": "2026-10-28T12:34:56.789012Z"}
```

3. GET — теперь не пустой:

```
curl http://127.0.0.1:8000/api/posts/
```

Ожидается: массив с одним постом.

4. GET — новые сверху: Создай второй пост. В списке первый должен быть **второй** (свежий) — проверь порядок `id` .

5. Пустой текст отклоняется:

```
curl -X POST http://127.0.0.1:8000/api/posts/ \
  -H "Content-Type: application/json" \
  -d '{"text":"   "}'
```

Ожидается `400 Bad Request` с `{"text": ["Пост не может быть пустым."]}` .

6. PUT/DELETE запрещены:

```
curl -X DELETE http://127.0.0.1:8000/api/posts/1/
```

Ожидается `405 Method Not Allowed` .

7. Browsable API: Открой `http://127.0.0.1:8000/api/posts/` в браузере. DRF покажет HTML-форму и список — можно создать пост прямо из браузера, что удобно для отладки.

8. Админка: `http://127.0.0.1:8000/admin/` → «Crackerchat → Посты» → список постов, поиск, удаление.

🚫 Подводные камни

- **`ModuleNotFoundError: No module named 'crackerchat'`** — приложение создано, но не добавлено в `INSTALLED_APPS` , либо ты создал его не из папки `backend/` . Проверь, что

`backend/crackerchat/apps.py` существует, и что запускаешь `manage.py` из `backend/`.

- **`no such table: crackerchat_post`** — не сделал `makemigrations + migrate` (или сделал только один из них).
- **`POST` через `curl` возвращает `403 CSRF verification failed`** — значит, не применил `DEFAULT_AUTHENTICATION_CLASSES: []` в `REST_FRAMEWORK` и не перезапустил сервер.
- **`POST` возвращает `415 Unsupported Media Type`** — забыл заголовок `-H "Content-Type: application/json"` в `curl`. (Для `fetch` на фронте браузер его ставит сам, если указать `Content-Type: application/json` в `headers`.)
- **Порядок в списке не тот** — проверь `Meta.ordering` в модели (`-created_at`, минус означает «по убыванию»).
- **`text` длиннее 2000 символов** — DRF вернёт `400` с ошибкой про `max_length`. Это ожидаемо.
- **POST через browsable API не работает** — для нашего `ViewSet` форма отображается, но для отправки `SessionAuthentication` не включена, поэтому форма попробует уйти без сессии. Проще тестировать через `curl` (см. проверку 2).

📝 Критерии готовности (Definition of Done)

- [ ] Приложение `crackerchat` создано и зарегистрировано в `INSTALLED_APPS`.
- [ ] Модель `Post` создана, миграция `0001_initial` сгенерирована и применена.
- [ ] `Post` зарегистрирован в админке, виден и редактируем.
- [ ] `PostSerializer` с `validate_text` (тримминг + отклонение пустого) готов.
- [ ] `PostViewSet` поддерживает только `list` и `create` (без update/destroy).
- [ ] Роутер подключён, `/api/posts/` доступен из `config/urls.py`.
- [ ] `REST_FRAMEWORK` настроен: `AllowAny`, без session-auth.
- [ ] `GET /api/posts/` возвращает массив (пустой или с постами, **новые сверху**).
- [ ] `POST /api/posts/` создаёт пост и возвращает `201` с `id` и `created_at`.
- [ ] `POST` с пустым `text` возвращает `400`.
- [ ] `DELETE /api/posts/1/` возвращает `405`.
- [ ] Browsable API доступен и работает для `GET`.
- [ ] Изменения закоммичены.